Lab#7

Structured Query Language (SQL)

Outline of Concepts

1. **Group By Statement**
2. **Having Clause**
3. **SQL Subqueries**
    - Concept
    - Syntax
    - Guidelines for Using Subqueries
    - Types of Subqueries

---

1. Group By Statement

The Group By statement groups rows that have same values into summary rows.

It is often used with aggregate functions to group the result set by one or more columns.

The aggregate functions include:

- COUNT(column_name) – Returns the number of rows in a group, ignoring NULLs.
- MIN(column_name) – Returns the smallest value in a column.
- MAX(column_name) – Returns the largest value in a column.
- SUM(column_name) – Returns the total sum of numeric values in a column.
- AVG(column_name) – Returns the average of numeric values in a column.
- STDEV(column_name) – Returns the standard deviation of values in a column.
- VAR(column_name) – Returns the variance of values in a column.
- STRING_AGG(column_name, delimiter) – Concatenates string values into a single string, separated by a specified delimiter.

You can find further documentation on aggregate functions on: https://learn.microsoft.com/en-us/sql/tsql/functions/aggregate-functions-transact-sql?view=sql-server-ver16

- **Command**:

```
SELECT column_name(s), AGGREGATE_FUNCTION(column_name)
FROM table_name
```

GROUP BY column_name(s)

- **Example**:

```
SELECT mode, SUM(amount) AS total
FROM payment
GROUP BY mode;
```

GROUP BY with Multiple Columns

SELECT department, job_title, COUNT(*) AS EmployeeCount

FROM Employees

GROUP BY department, job_title;

This groups employees by department and job title.

---

2. HAVING Clause

The **HAVING** clause is used to apply a filter on the result of **GROUP BY** based on the specified condition.

The **WHERE** clause places conditions on the **selected** columns, whereas the **HAVING** clause places conditions on groups created by **GROUP BY** clause

- **Command**:

SELECT column_name(s)

FROM table_name

WHERE condition(s)

GROUP BY column_name(s)

HAVING condition(s)

- **Example**:

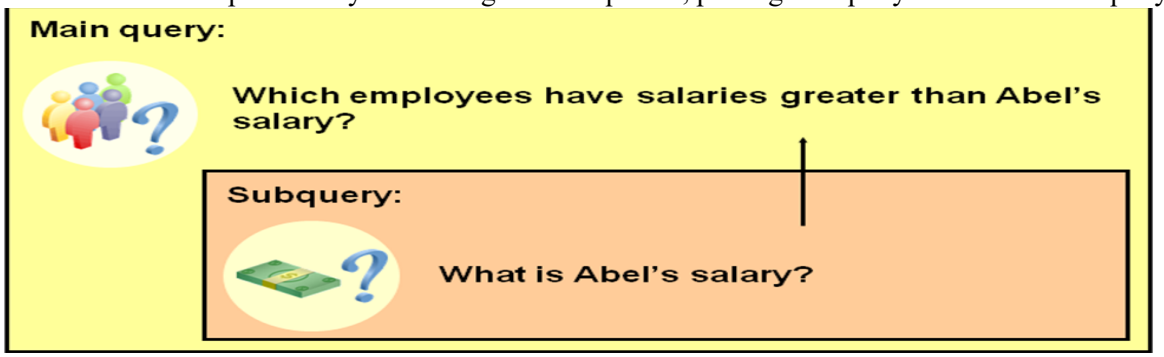
```
SELECT mode, COUNT(amount) AS total
FROM payment
GROUP BY mode
HAVING COUNT(amount)>=3
ORDER BY total ASC;
```

---

3. SQL Subqueries

Suppose you want to write a query to find out who earns a salary greater than Abel‘s salary. To solve this problem, you need two queries: one to find how much Abel earns, and a second query to find who earns more than that amount.

You can solve this problem by combining the two queries, placing one query inside the other query.



The inner query (or subquery) returns a value that is used by the outer query (or main query). Using a subquery is equivalent to performing two sequential queries and using the result of the first query as the search value in the second query.

Syntax (Sub-Query):

A subquery is a SELECT statement that is embedded in the clause of another SELECT statement. You can build powerful statements out of simple ones by using subqueries. They can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself.

You can place the subquery in a number of SQL clauses, including the following:

- WHERE clause.
- HAVING clause.
- FROM clause.

SELECT <*columns*>

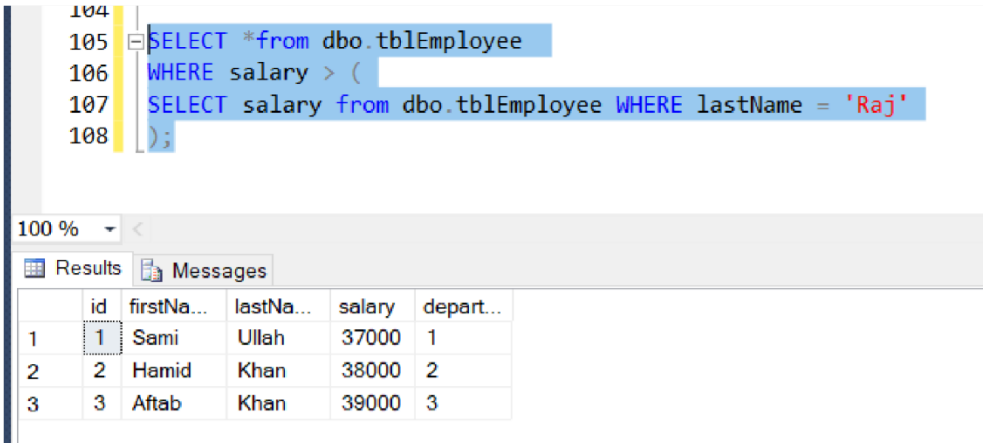
FROM table
WHERE expression *operator* (SELECT <*columns*> FROM table);
In the above syntax: *operator* includes a comparison condition such as >, =, or IN

Note: Comparison conditions fall into two classes: single-row operators (>, =, >=, <, <>, <=) and multiple-row operators (IN, ANY, ALL). The subquery is often referred to as a nested SELECT, sub-SELECT, or inner SELECT statement. The subquery generally executes first, and its output is used to complete the query condition for the main (or outer) query.

Exercise:

Using subquery, select record from *tblEmployee* whose salary is greater than the record whose *lastName* is **Raj.**

Please practice it.



In the example, the inner query determines the salary of employee **Raj** that is 35000. The outer query takes the result of the inner query and uses this result to display all the employees who earn more than employee **Raj**.

Guidelines for Using Subqueries:

- A subquery must be enclosed in parentheses.
- Place the subquery on the right side of the comparison condition for readability however, the subquery can appear on either side of the comparison operator.
- Two classes of comparison conditions are used in subqueries: single-row operators and multiple-row operators.

Types of Subqueries:

Single-Row Subqueries

Return only one row

Use single-row comparison operators

Operator	Meaning
=	**Equal to**
>	**Greater than**
>=	**Greater than or equal to**
<	**Less than**
<=	**Less than or equal to**
<>	**Not equal to**

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name FROM table_name WHERE condition);
```

Multiple-Row Subqueries

- **Return more than one row**
- **Use multiple-row comparison operators**

Operator	Meaning
IN	Equal to any member in the list
ANY	Compare value to each value returned by the subquery
ALL	Compare value to every value returned by the subquery

```
SELECT column_name

FROM table_name

WHERE column_name IN (SELECT column_name FROM table_name WHERE condition);
```

Correlated Subqueries

A correlated subquery is one that refers to a column in the outer query. It is evaluated once for each row processed by the outer query.

```
SELECT column_name

FROM table_name outer_alias

WHERE column_name operator (SELECT column_name FROM table_name WHERE
outer_alias.column_name = column_name);
```

Subqueries in Where Clause

```
SELECT emp_name, salary

FROM Employees

WHERE salary > (

SELECT AVG(salary)

FROM Employees

);
```

Subqueries in From Clause

```
SELECT dep_name, AVG(salary) AS avg_salary

FROM (

SELECT dep_name, salary

FROM Employees

) AS TempTable
```

Subqueries in HAVING Clause

```
SELECT dep_name, SUM(salary) AS total_salary

FROM Employees

GROUP BY dep_name

HAVING SUM(salary) > (

SELECT AVG(total_salary)

FROM (

SELECT SUM(salary) AS total_salary

FROM Employees

GROUP BY dep_name

) AS DeptTotalSalaries

);
```